

Augmenting Machine Learning with Genetic AI

A Hybridisation Approach: Layers, Attention, and Open Directions

Summary. We investigate whether the inductive biases of Genetic AI, specifically the fixed-point fitness computation that models population-level gene competition, can enrich standard machine learning components. Rather than replacing gradient-based learning, we treat genetic fitness as a structural prior that can be hybridised into neural networks at different levels of granularity. This white paper reports two completed experiments and two open research directions. *Experiment I* hybridises genetic fitness into feed-forward and multi-head layers across seven tabular classification benchmarks. *Experiment II* modulates value projections in BERT attention with genetic fitness for dense passage retrieval on MS-MARCO. The central finding is that hybridisation succeeds in *parallel* multi-head topologies, where genetic competition acts as an ensemble prior, but degrades in deep sequential stacks. Genetic priors also accelerate early training dynamics before saturating at convergence, pointing to adapter and routing architectures as the most promising future direction.

1. Introduction

Genetic AI can be embedded in neural networks as a parameter-free structural prior that *augments* gradient-based learning rather than replacing it. Its core computation maps a population matrix $\mathbf{P} \in [0, 1]^{n_{\text{org}} \times n_{\text{gene}}}$ — derived from neural activations via a sigmoid projection — to per-gene and per-organism fitness scores using a fixed formula. The fitness step has no learnable parameters; the surrounding projection weights are trained normally with standard optimisers.

This document reports the findings of two hybridisation experiments and proposes two further directions, ordered by the architectural component targeted:

1. **Genetic Layers** (§3): replacing feed-forward and multi-head linear transformations with GeneticLayer modules across seven tabular classification benchmarks.
2. **Genetic Attention** (§4): modulating value projections in BERT attention with genetic fitness for dense passage retrieval.
3. **Genetic Adapter** (§5): proposed multi-task classification on a frozen encoder.
4. **Genetic MoE** (§5): proposed fitness-based expert routing in MoE architectures.

2. Background: Genetic Fitness as a Structural Prior

Population Matrix and Fitness Computation

The genetic prior is expressed through a *population matrix* $\mathbf{P} \in [0, 1]^{n_{\text{org}} \times n_{\text{gene}}}$, obtained from activations $\mathbf{x} \in \mathbb{R}^d$ via a sigmoid projection $\sigma(W\mathbf{x} + \mathbf{b})$ reshaped to $(n_{\text{org}}, n_{\text{gene}})$. Each sample in a batch produces a separate matrix: rows are organisms (latent feature groups within that sample) and columns are genes (latent dimensions). Gene fitness is

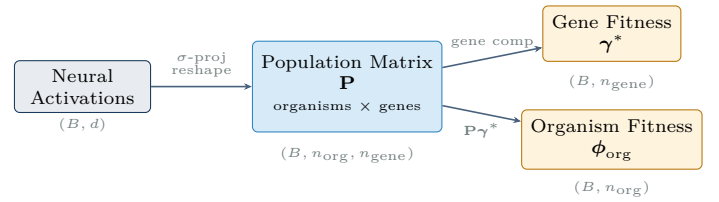


Figure 1: Genetic AI transforms neural activations into a population matrix where rows represent organisms (latent feature groups within each sample) and columns represent genes (latent dimensions). For each sample in the batch, a separate population matrix is created. Fixed-formula fitness computations yield gene fitness through relative competition and organism fitness through weighted gene expression.

then computed as:

$$\tilde{\phi}_g = \frac{1}{n_{\text{org}}} \sum_{i=1}^{n_{\text{org}}} \mathbf{P}_{ig}, \quad (1)$$

$$\gamma_g^* = \left(\sum_{s=1}^{n_{\text{gene}}} \frac{\tilde{\phi}_g + \frac{1}{2}}{\tilde{\phi}_s + \frac{1}{2}} \right)^{-1}. \quad (2)$$

Organism fitness follows as:

$$\phi_{\text{org}} = \mathbf{P} \gamma^*. \quad (3)$$

The fitness computation is parameter-free: no weights are learned beyond the projection matrices that produce $\mathbf{P}$. The $+\frac{1}{2}$ offset prevents division by zero and moderates competition when genes are similarly expressed. The surrounding gradient-trained layers determine what goes into $\mathbf{P}$. Figure 1 illustrates the full pipeline.

3. Experiment I: Genetic Layers

Background

A standard feedforward layer maps input $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ through a learned linear transformation followed by a non-linearity σ :

$$\mathbf{h} = \sigma(W\mathbf{x} + \mathbf{b}), \quad W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}, \mathbf{b} \in \mathbb{R}^{d_{\text{out}}}.$$

A multi-head network runs K independent heads in parallel and concatenates their outputs via an output projection W_O :

$$\text{MH}(\mathbf{x}) = [\mathbf{h}_1(\mathbf{x}) \parallel \dots \parallel \mathbf{h}_K(\mathbf{x})] W_O.$$

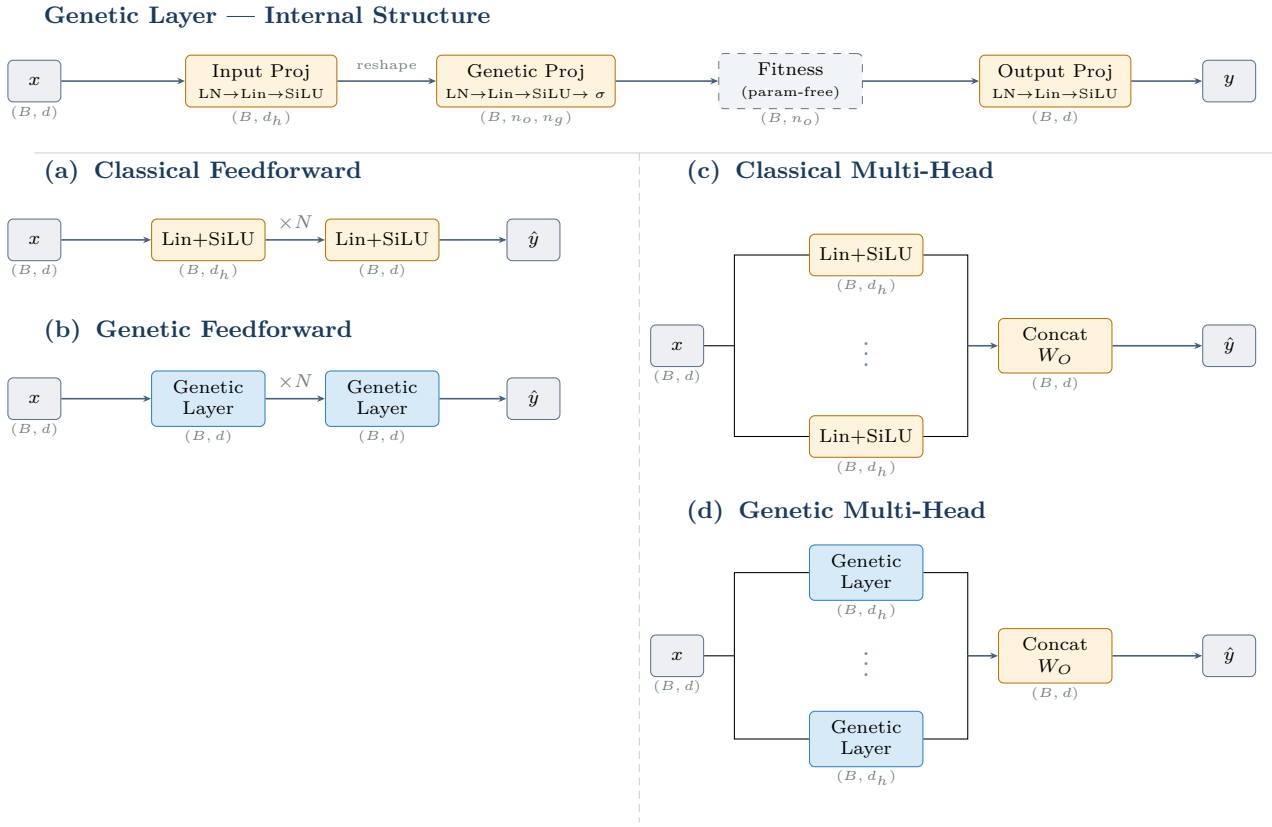


Figure 2: *Top:* internal structure of a Genetic Layer (Section 3). Amber boxes are learned projections (LayerNorm $\rightarrow$ Linear $\rightarrow$ SiLU); dashed grey box is the parameter-free fitness computation. The Genetic Projection outputs a flat vector that is reshaped into the population matrix structure. *Bottom:* the four topology variants evaluated in Experiment I. Amber boxes are all learned blocks (linear layers and output projections); blue boxes are Genetic Layer modules (each expands $(B, d) \rightarrow (B, n_o, n_g) \rightarrow (B, n_o)$ then projects back to (B, d)). Labels show output tensor shape (B = batch, d = feature dim, d_h = head/hidden dim, n_o = organisms, n_g = genes).

This experiment replaces classical linear transformations with **GeneticLayer** modules that sandwich the parameter-free fitness computation between learned projections. An input projection (LayerNorm $\rightarrow$ Linear $\rightarrow$ SiLU) maps the input to a hidden dimension. A genetic projection (LayerNorm $\rightarrow$ Linear $\rightarrow$ SiLU $\rightarrow \sigma$) maps that to the population matrix $\mathbf{P} \in [0, 1]^{n_{\text{org}} \times n_{\text{gene}}}$. Fitness is then computed via Eqs. (1)–(3), and an output projection maps organism fitness scores to the target dimension. Figure 2 shows the internal structure and the four topology variants.

Setup

This experiment evaluates genetic layers as substitutes for learned linear transformations on seven tabular classification benchmarks. We replace the linear layers in two standard network topologies with hybrid *genetic layers*: each layer projects its input to a population matrix via a sigmoid, applies the fitness computation of Eqs. (1)–(3), and returns the organism fitness vector. The surrounding projection weights are trained with Adam; the fitness computation itself is parameter-free. Seven tabular datasets are used (Iris, Wine, Breast Cancer, Digits, Synthetic, CovType, Olivetti Faces), spanning 4 to 4,096 features. For CovType, a 10,000-sample subset of the full 581K-sample dataset was used. Two topologies are evaluated: *feedforward* stacks (depths 2, 4, 6) and *parallel heads* (1, 2, 3 heads concatenated). All configurations: 128 epochs, Adam optimiser, batch size 16,384. The evaluation metric is balanced accuracy.

Results

Table 1: Average performance across all seven datasets.

Topology	Type	Bal. Acc.	Params
Feedforward	Classical	88.7%	48,851
Feedforward	Genetic	49.1%	25,667
Heads	Classical	79.5%	41,330
Heads	Genetic	80.8%	44,235

The topological split is stark and consistent. Genetic feedforward networks underperform by 39.6 percentage points on average and, crucially, *degrade with depth*. On Digits, accuracy falls from 53.0% at depth 2 to 29.4% at depth 4 and 18.8% at depth 6 — the inverse of the classical trend (94.8%, 97.2%, 97.5%). The same pattern holds on every high-dimensional dataset. Because Eq. (2) is a fixed-formula transformation, stacking multiple such layers does not build abstraction; it compounds information loss at each stage.

Genetic multi-head networks outperform classical heads on average (80.8% vs. 79.5%) and scale better with head count. Classical networks show only moderate improvement from 1 to 3 heads; genetic networks improve substantially. Table 2 shows the three datasets with the largest genetic advantage.

We speculate that parallel genetic heads succeed because each head maps the input to a distinct population matrix $\mathbf{P}$, inducing an ensemble of complementary fitness landscapes. The three top-performing datasets share medium feature

Table 2: Datasets where genetic heads most exceed classical heads (best genetic head count vs. best classical head count).

Dataset	Genetic	Classical	Δ
Synthetic (50 feat.)	59.6%	47.6%	+12.0pp
CovType (54 feat.)	75.0%	66.6%	+8.4pp
Digits (64 feat.)	97.6%	94.8%	+2.8pp

dimensionality (50–64) and multi-class output structure (7, 7, and 10 classes). By contrast, a single deep stack repeatedly applies the same fixed-formula transformation and compounds information loss toward a fixed-point.

Parameter efficiency is competitive in the multi-head setting: genetic heads use only 7% more parameters than classical on average, compared to 48% *fewer* for genetic feedforward (which achieves far less accuracy). Training overhead is notable: 17% slower for genetic heads and 27% slower for genetic feedforward.

Limitations

These are preliminary results. Each configuration used a single fixed train/test split (80/20, `random_state=8080`), with no cross-validation or repeated runs. For small datasets such as Iris ($n=150$) and Wine ($n=178$), a single split can yield high-variance estimates, and reported differences may not survive a change of seed. The depth-degradation finding is consistent across all seven datasets and likely robust. The per-dataset multi-head figures in Table 2 are indicative only.

Key Takeaway

Genetic AI hybridisation is **topology-sensitive**: it fails in sequential depth stacks and succeeds in parallel multi-head configurations. Hybrid multi-head networks match or outperform fully-learned counterparts on several medium-dimensional multi-class benchmarks at a comparable parameter budget.

4. Experiment II: Genetic Attention

Background

Multi-Head Attention (MHA) [?] maps the input $X \in \mathbb{R}^{B \times T \times d}$ (batch B , sequence length T , hidden size d) to queries, keys, and values $Q, K, V \in \mathbb{R}^{B \times T \times d_k}$ via learned projections $W^Q, W^K, W^V \in \mathbb{R}^{d \times d_k}$, with $d_k = d/h$ for h heads:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V.$$

The attention output is the value-weighted sum over all positions, with weights derived from scaled query–key similarity:

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

All projection matrices are fully learned. The parallel-head success in Experiment I motivates asking whether a genetic prior on the value pathway can improve retrieval, keeping the QK similarity computation intact.

Design

Initial decoder designs explored two strategies for injecting genetic fitness into attention:

- **Value-only:** Q and K are eliminated. The value matrix $V \in \mathbb{R}^{B \times h \times T \times d_k}$, scaled to $[0, 1]$, is used directly as

the population matrix $\mathbf{P}$ (organisms = tokens, genes = d_k head features). Eqs. (1)–(2) yield gene fitness $\gamma^* \in \mathbb{R}^{d_k}$; the score $\phi_{ij} = V_j \cdot \gamma_i^*$ replaces the usual $Q_i \cdot K_j$ similarity.

- **Score-matrix modulation:** a $T \times T$ score matrix S is constructed (either from dedicated projections $O'G'^\top/\sqrt{d_k}$ replacing QK entirely, or as $\sigma(QK^\top/\sqrt{d_k})$ on top of the standard path). S is used as the population matrix (organisms = query tokens, genes = key tokens); Eqs. (1)–(3) yield per-token organism fitness $\phi_{\text{org}} \in \mathbb{R}^T$, which re-weights the rows of S before softmax and multiplication by V .

Neither approach converged reliably in the decoder setting. Two compounding failure modes were identified:

Statistical instability. The gene-mean computation (Eq. (1)) estimates each gene’s typical expression level by averaging normalised value activations across all sequence positions. In a decoder with causal masking, position t can only see positions $1, \dots, t$, so early tokens average over just one or two values, which is an extremely noisy estimate. Because each position sees a different prefix length, $\tilde{\Phi}_g$ is also asymmetric across positions, making it an unstable learning signal during early training. The genetic prior is intended to be a stable structural signal; in the causal setting it becomes a moving, position-dependent quantity.

Computational cost. A correct causal implementation cannot compute a single shared gene-fitness vector per forward pass. To preserve causality, the fitness computation must be run separately for each prefix length $t = 1, \dots, T$, producing T independent γ^* vectors before the attention output at any position can be formed. This makes the per-step cost scale as $O(T)$ gene-fitness evaluations rather than one, an overhead that compounds the instability problem and makes training impractically slow at even moderate sequence lengths.

Final Design

Both failure modes trace back to a fundamental structural mismatch: the gene-mean computation requires a stable average over *all* visible positions, but causal masking makes that set grow with each token, rendering the mean noisy and position-dependent. The genetic prior is therefore better suited to *non-causal* (bidirectional) attention, where every position sees the full sequence and $\tilde{\Phi}_g$ is stable. We therefore applied the design to a BERT-style encoder, keeping the standard Q–K path intact and applying the genetic prior only to V . Unlike the Genetic Layer (Figure 2), which adds dedicated projections around the fitness computation, this approach applies fitness directly to the value matrix V produced by standard attention, without introducing new parameters. We call the resulting mechanism Multi-Head

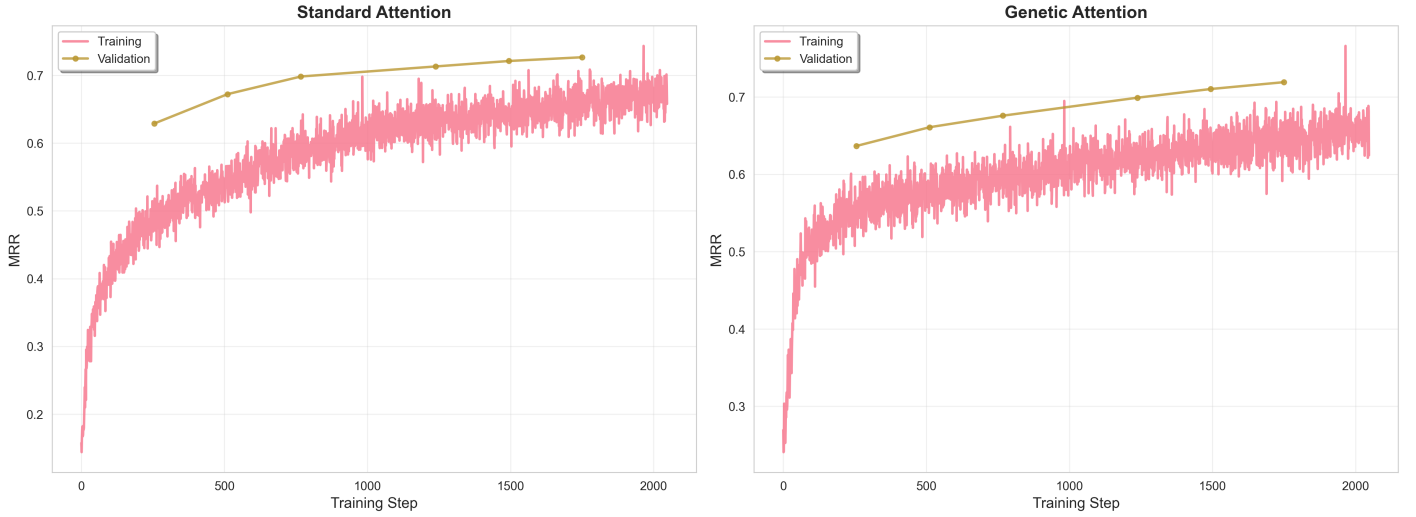


Figure 3: Mean Reciprocal Rank over training steps. Genetic attention leads early before being overtaken by the standard baseline, which continues improving to convergence.

Genetic Attention (MGA):

$$V_{\text{norm}} = \sigma(V), \quad (4)$$

$$\tilde{\Phi}_g = \frac{1}{T} \sum_{t=1}^T V_{\text{norm}}[:, :, t, g], \quad (5)$$

$$\gamma_g^* = \left(\sum_{s=1}^d \frac{\tilde{\Phi}_g + \frac{1}{2}}{\tilde{\Phi}_s + \frac{1}{2}} \right)^{-1}, \quad (6)$$

$$V_{\text{gen}} = V \odot \gamma^*, \quad (7)$$

$$\text{MGA}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V_{\text{gen}}. \quad (8)$$

This modulates *which features* the attention output emphasises, without altering *which positions* it weights. The fitness γ^* is computed fresh for each forward pass from the value activations of the current batch, making it input-dependent and parameter-free beyond the existing value projection weights.

Setup

A six-layer BERT [?] (hidden size 384, 12 attention heads, $d_{\text{head}} = 32$, intermediate size 1,536) is trained as a dual encoder [?] for passage retrieval on MS-MARCO [?] (503K training pairs, 101K test pairs). Query and passage encoders share the same backbone. Sentence representations are obtained by mean pooling over valid tokens and L2-normalising. Training uses InfoNCE loss with in-batch negatives (batch size 512), AdamW with linear warmup, fixed seed 8080, for 2048 steps.

Results

Table 3: MS-MARCO dual-encoder results at 2,048 training steps.

Config	MRR	NDCG@5	Hit@5	Time (s)
Standard	0.492	0.500	0.573	7,822
Genetic	0.486	0.492	0.568	11,947
Δ	-1.2%	-1.6%	-0.9%	+52.8%

At 2,048 steps, standard attention outperforms genetic attention across all retrieval metrics. The training curves

(Figure 3) show an early-training reversal: genetic attention *leads* during the first several hundred steps before the standard baseline overtakes it and continues to improve. This is consistent with γ^* acting as an initialisation prior. Once the value projections are sufficiently trained, unconstrained attention has greater capacity and the fixed modulation becomes a ceiling.

The 52.8% training overhead arises because γ^* is not a fixed parameter: it is derived from the value activations of the current batch on every forward pass. Concretely, Eqs. (1)–(2) run as an extra sequential step after every value projection.

Limitations

These results should be treated as preliminary: one seed, one run per configuration, and a 2,048-step budget. No repeated runs, hyperparameter sweeps, or significance testing were performed. A rigorous comparison would require multiple seeds, runs to full convergence, and paired statistical tests. The reported metric differences (≤ 1.6 percentage points) are within a range explainable by seed variance alone.

Hybridising attention with a genetic value prior provides an **early-training advantage** in retrieval metrics but is overtaken at convergence. Fully-learned attention achieves lower validation loss and better test metrics with 35% less training time.

5. Future Directions

Experiment III: Genetic Adapter

We propose attaching a genetic multi-head module to a frozen BERT encoder as a lightweight classification adapter, motivated by the multi-head gains in Experiment I. The evaluation target is the Bias-in-Bios dataset [?] (biography text, profession and gender labels). The multi-task structure tests whether independent genetic heads partition fitness landscapes by task. Counterfactual pronoun-swapping would additionally test whether genetic competition incidentally reduces gender bias. Figure 4 shows the proposed

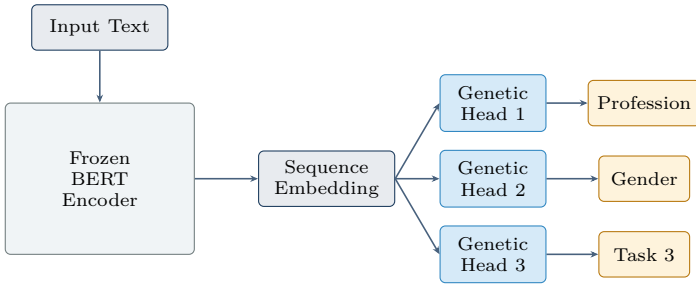


Figure 4: Proposed Genetic Adapter architecture: independent genetic heads on a frozen encoder partition fitness landscapes by task.

architecture.

Experiment IV: Genetic Expert Routing in MoE

We propose replacing the learned router in MoE architectures [?] with a genetic fitness computation: each expert is treated as an organism, each token’s feature dimensions as genes, and the top- k experts are selected by organism fitness (3). This yields a parameter-free routing mechanism analogous to the multi-head ensemble effect in Experiment I, and may avoid the load-balancing instabilities of learned routers. Validating this requires at least $\sim 1\text{B}$ parameter models and is deferred to higher-compute settings. Figure 5 illustrates the proposed routing mechanism.

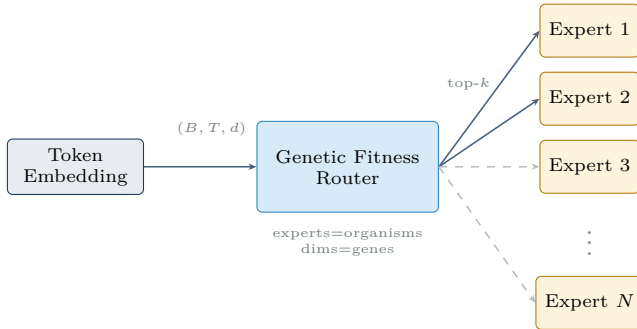


Figure 5: Proposed Genetic MoE architecture: organism fitness scores replace learned routing. Top- k selection by fitness provides parameter-free, diversity-preserving routing.

6. Conclusion

The two completed experiments establish a clear boundary condition for Genetic AI hybridisation with machine learning. First, hybridisation is *topology-sensitive*: inserting the genetic prior into sequential depth stacks compounds information loss at each fixed-formula stage, causing performance to degrade monotonically with depth. By contrast, parallel multi-head architectures benefit from the genetic prior because each head converges to a distinct fitness equilibrium, producing a complementary ensemble. Second, genetic priors *accelerate early training but saturate*: genetic value modulation leads retrieval metrics early before the unconstrained baseline catches up at convergence.

The genetic prior is most useful when *diversity is incentivised* (parallel independent heads) and *training is short* (warm-up or cold-start settings). It becomes a constraint in sequential stacks and long fine-tuning runs, where the fixed formula limits capacity. The proposed Genetic Adapter and Genetic MoE experiments target larger scale and more complex

task structure, to test whether gains extend beyond the early-training window.

Code and results are available in danube.ai’s open-source repository: <https://github.com/danube-ai/pikaia>.

Acronyms

BERT Bidirectional Encoder Representations from Transformers

MGA Multi-Head Genetic Attention

MHA Multi-Head Attention

MoE Mixture of Experts

NDCG Normalised Discounted Cumulative Gain